Algoritmos

Baseado na disciplina “Modelos de Predição em Saúde”
ministrada pelo Professor Alexandre Chiavegatto Filho (FSP/USP)

Nádia Cristina Pinheiro Rodrigues

Algoritmos baseados em árvores

- Random Forest
- XGBoost
- CatBoost
- LightGBM

Como esses algoritmos funcionam

Árvore de decisão

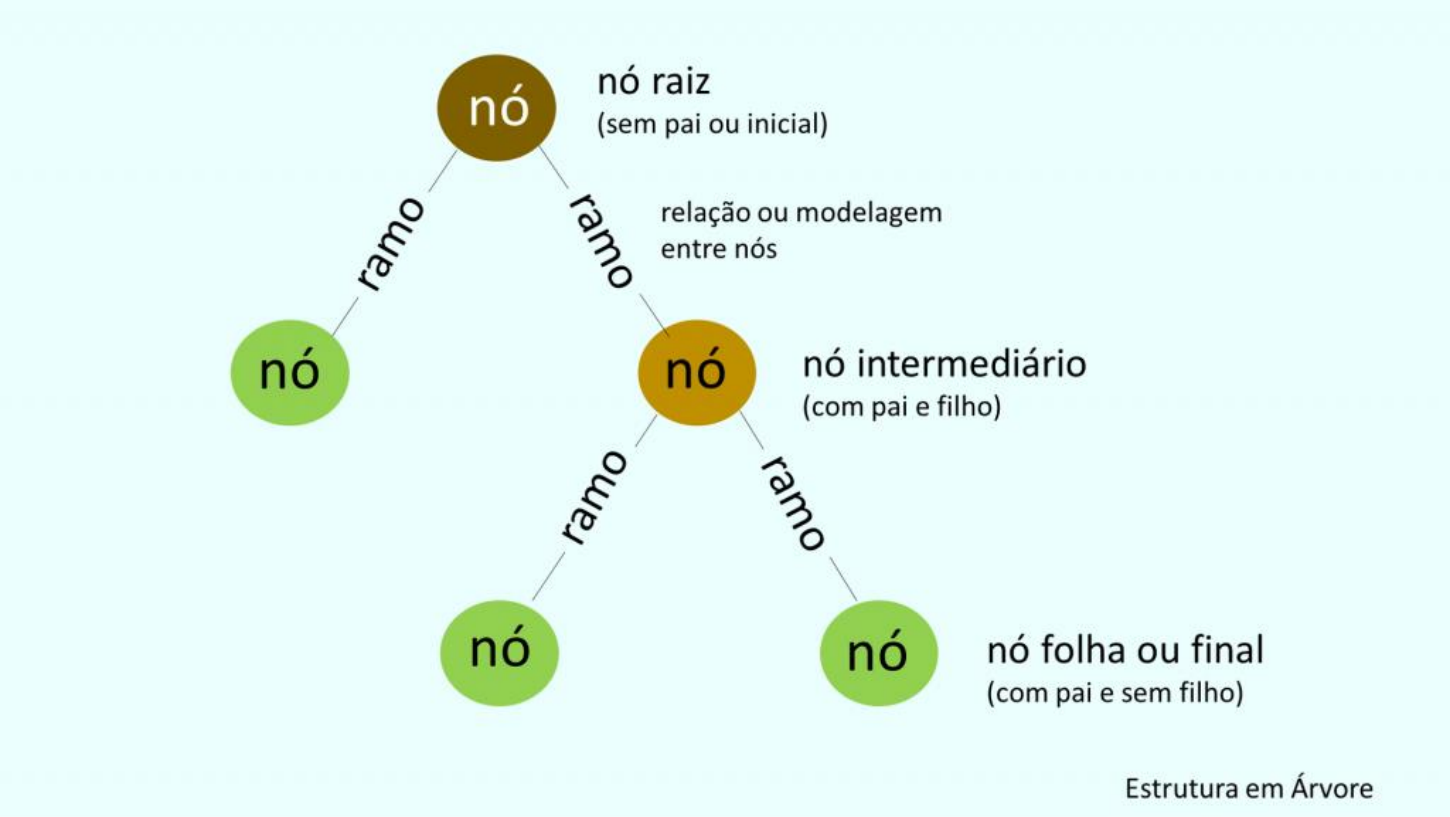
Semelhante a árvore

Nós -> variáveis

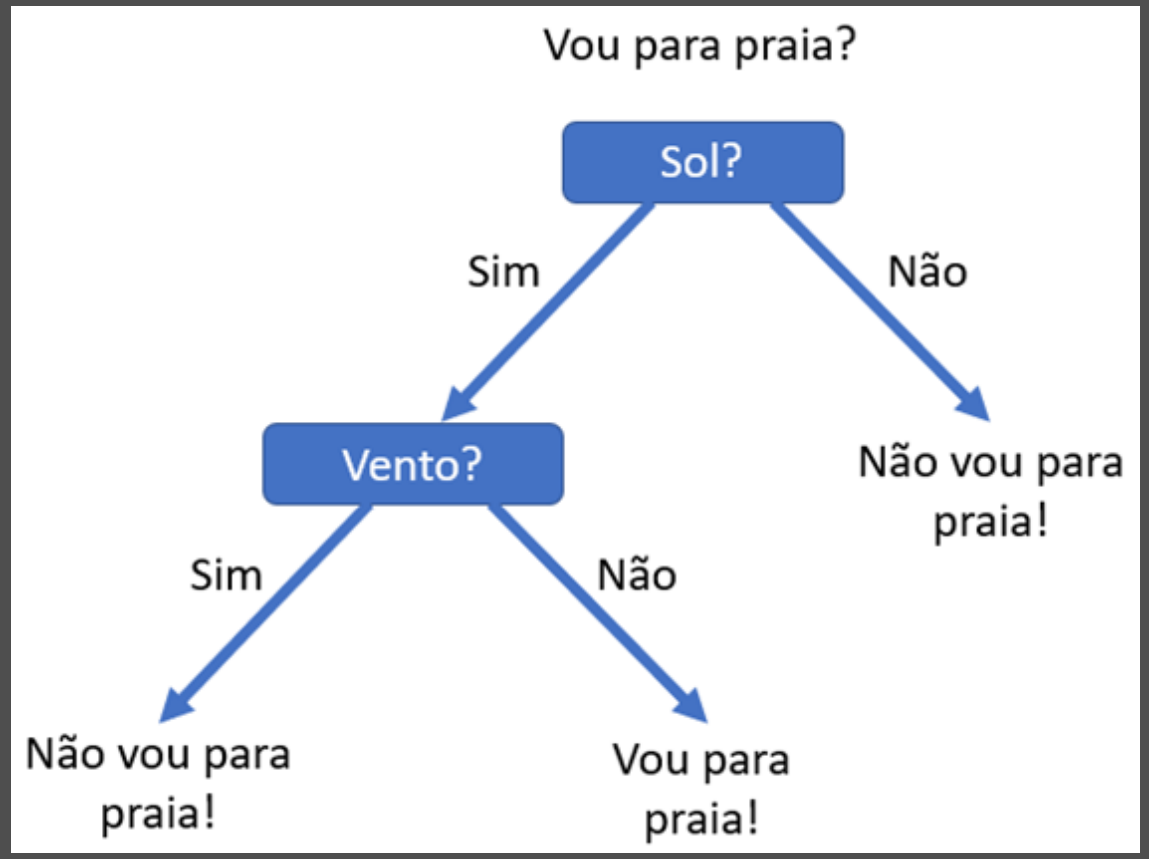
Ramos dividem os dados em grupos

O número de OBS é atribuído a cada nó

A folha representa a predição final



#MachineLearning
Vou assistir aula na universidade hoje?
Tenho aula marcada?
Sim
Não
Não vou
A aula é remota?
Sim
Não
Não vou
Eu tenho como ir para a aula no meu carro?
Sim
Não
Vou
Da tempo ir de Transporte público?
Sim
Não
Vou
Não vou
@ramondomingos
Ramon Domingos
ÁRVORE DE DECISÃO


Vou para praia?
Sol?
Sim
Não
Vento?
Não vou para praia!
Sim
Não
Não vou para praia!
Vou para praia!

Ensemble

O caso de **árvore de decisão simples**:

Vantagem: fácil de interpretar

Desvantagem: usar apenas uma árvore de decisão pode levar ao sobreajuste do modelo

A solução é o **ensemble**, que permite maior generalização

Ensemble => combinar várias árvores

“árvores de decisão em conjunto” (ensemble de árvores)

Existem diferentes formas de combinar as árvores

Os algoritmos de boosting são uma estratégia de combinação e o Random Forest também.

Ensemble

Exemplo TB: Cura ou abandono?

REALIDADE	ABANDONO	CURA	CURA	ABANDONO	ABANDONO	CURA	CURA
PREDITOR 1	ABANDONO	ABANDONO	CURA	CURA	ABANDONO	CURA	ABANDONO
PREDITOR 2	CURA	CURA	CURA	CURA	ABANDONO	CURA	ABANDONO
PREDITOR 3	ABANDONO	CURA	ABANDONO	ABANDONO	CURA	ABANDONO	CURA
PREDITOR 4	ABANDONO	CURA	ABANDONO	ABANDONO	CURA	CURA	CURA
PREDITOR 5	CURA	ABANDONO	CURA	ABANDONO	ABANDONO	ABANDONO	CURA
COMITÊ	ABANDONO	CURA	CURA	ABANDONO	ABANDONO	CURA	CURA

Algoritmos baseados em árvores

Bagging (modelos paralelos, comitê de votação):

- Combinação de múltiplas árvores de decisão (usa todos os atributos disponíveis)
- Random Forest (seleção aleatória de atributos)

Boosting (modelos sequenciais com foco em reduzir os erros do modelo anterior)

- XGBoost
- CatBoost
- LightGBM

Bagging de Árvores de Decisão (Múltiplas Árvores de Decisão)

Utiliza amostragem bootstrap: amostras aleatórias dos dados de treino com reposição.

Cada amostra bootstrap é usada para treinar uma árvore de decisão.

As árvores são complexas (sem regularização/poda).

Técnica de ensemble, combinando várias árvores:

- Classificação → classe mais votada (> proporção de votos).

Ao combinar várias árvores de decisão, os erros específicos de cada uma tendem a se compensar. Isso torna o modelo final menos sensível a variações dos dados de treinamento, reduzindo o sobreajuste e melhorando o desempenho em novos dados.

Bagging de Árvores de Decisão (Múltiplas Árvores de Decisão)

Árvores são independentes, permitindo paralelização.

Pode envolver centenas ou milhares de árvores.

Custo computacional elevado e tempo de treinamento alto.

Perda de interpretabilidade em relação a uma única árvore.

Todos os preditores são usados em todos os nós, o que pode:

- Dar peso excessivo a alguns atributos
- Gerar árvores muito semelhantes

Random Forest

Bagging + seleção de preditores

Random Forest é essencialmente **Árvores de Decisão**, **com adição de aleatoriedade nas features**.

Redução da correlação entre árvores

Ao contrário da Árvore de Decisão, que usa **todas as features em seus nós**, o Random Forest seleciona **um subconjunto aleatório de preditores.**

Isso garante que as árvores sejam mais **diversificadas** e **menos correlacionadas**, aumentando a robustez do ensemble.

Bagging sozinho treina árvores complexas, mas elas podem ficar muito parecidas.

A seleção aleatória de variáveis ajuda a **diminuir essa semelhança**.

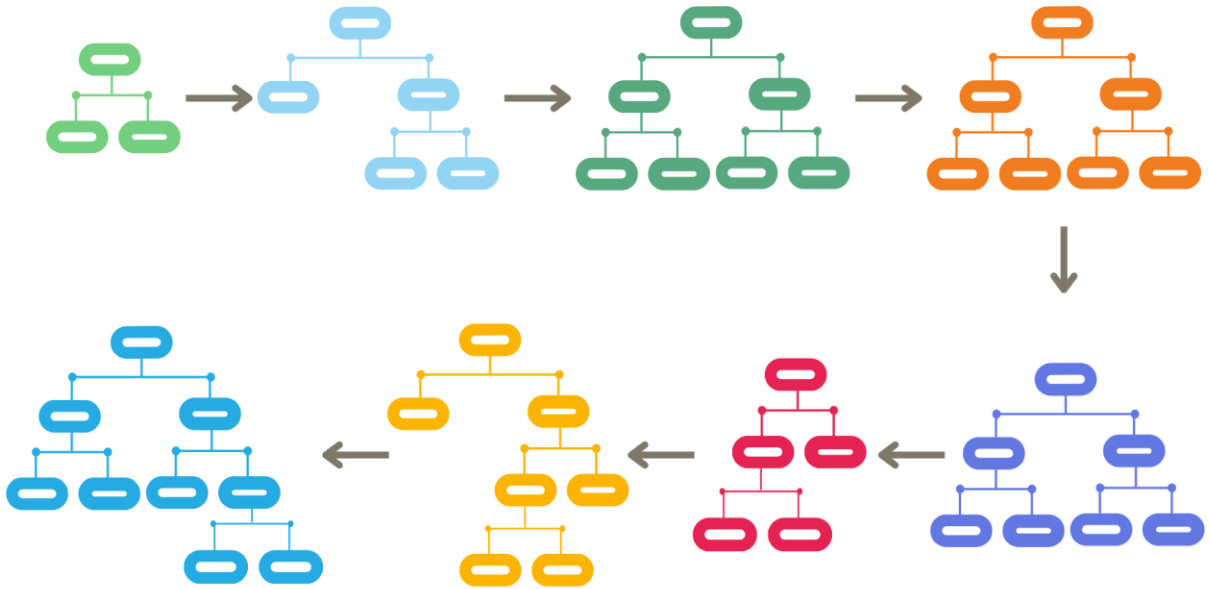
Regularização via número de preditores

O **principal parâmetro de regularização** do Random Forest é quantos preditores são avaliados em seus nós.

Menos preditores → árvores mais diversas → menor correlação → ensemble mais estável.

Algoritmos de Boosting

Treina árvores sequenciais, cada uma tentando corrigir os erros anteriores



Algoritmos de Boosting

Processo Sequencial: Diferente do bagging (que treina árvores em paralelo), o boosting treina as árvores de forma sequencial. Cada nova árvore é um "aprendiz fraco". No boosting, em vez de criar uma única árvore gigante e complexa, usam-se milhares dessas árvores pequenas que, sozinhas, seriam inúteis, mas juntas tornam-se um aprendiz forte.

Aprendizagem Aditiva: O modelo final é uma função aditiva, onde a previsão final é a combinação cumulativa (soma ponderada) da primeira árvore e de todas as correções feitas pelas árvores seguintes.

Algoritmos de Boosting

Treinamento iterativo -> vai construindo as árvores uma por uma

Objetivo do treinamento: Minimizar uma função de erro, ou seja, criar o melhor modelo possível

Exemplo:

Regressão → erro quadrático médio (MSE)

Classificação → AUC-ROC

Cada nova árvore é treinada para corrigir os erros do modelo anterior. A cada etapa, o modelo ajusta suas previsões para reduzir esse erro, somando progressivamente as contribuições das árvores e melhorando o desempenho final.

Algoritmos de Boosting

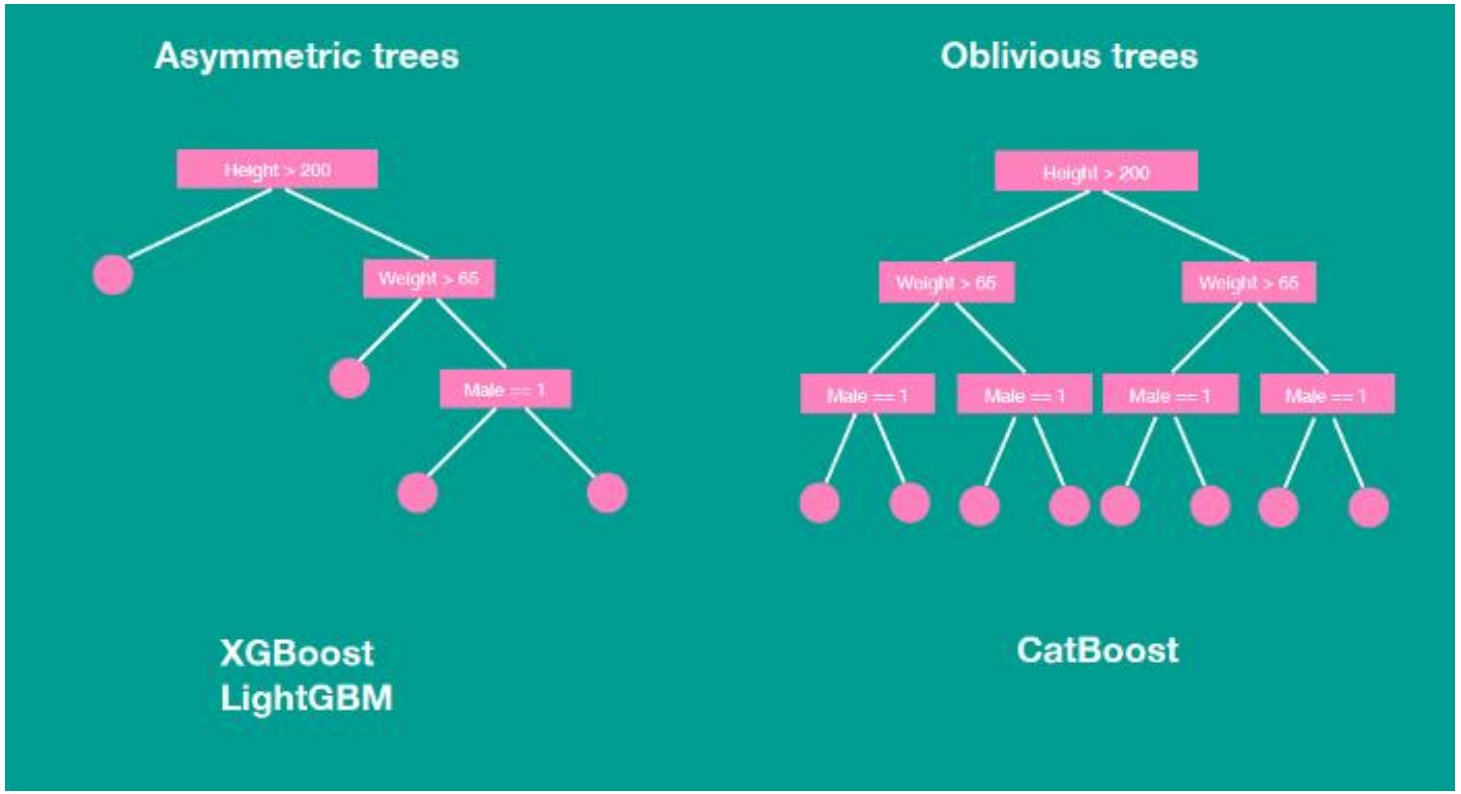
Todo processo é feito por meio de divisão repetitiva dos dados até atingir algum limite

- Cada árvore divide os dados em **nós de decisão** baseados em features
- Isso é feito **repetidamente**, construindo a árvore até atingir algum critério (como número máximo de divisões ou profundidade máxima).
- Essas divisões buscam **maximizar algum critério de qualidade** da árvore, como ganho de informação ou redução de variância.
- Para evitar **overfitting**, cada árvore é geralmente **pequena e rasa.**
- Limites típicos:
  - Profundidade máxima da árvore
  - Número mínimo de exemplos por folha
  - Número máximo de folhas

Algoritmos de Boosting

Árvores assimétricas tendem a ser mais flexíveis, podem capturar relações complexas, mas são mais difíceis de interpretar e otimizar.

Árvores simétricas simplificam a estrutura da árvore, facilitando a regularização.



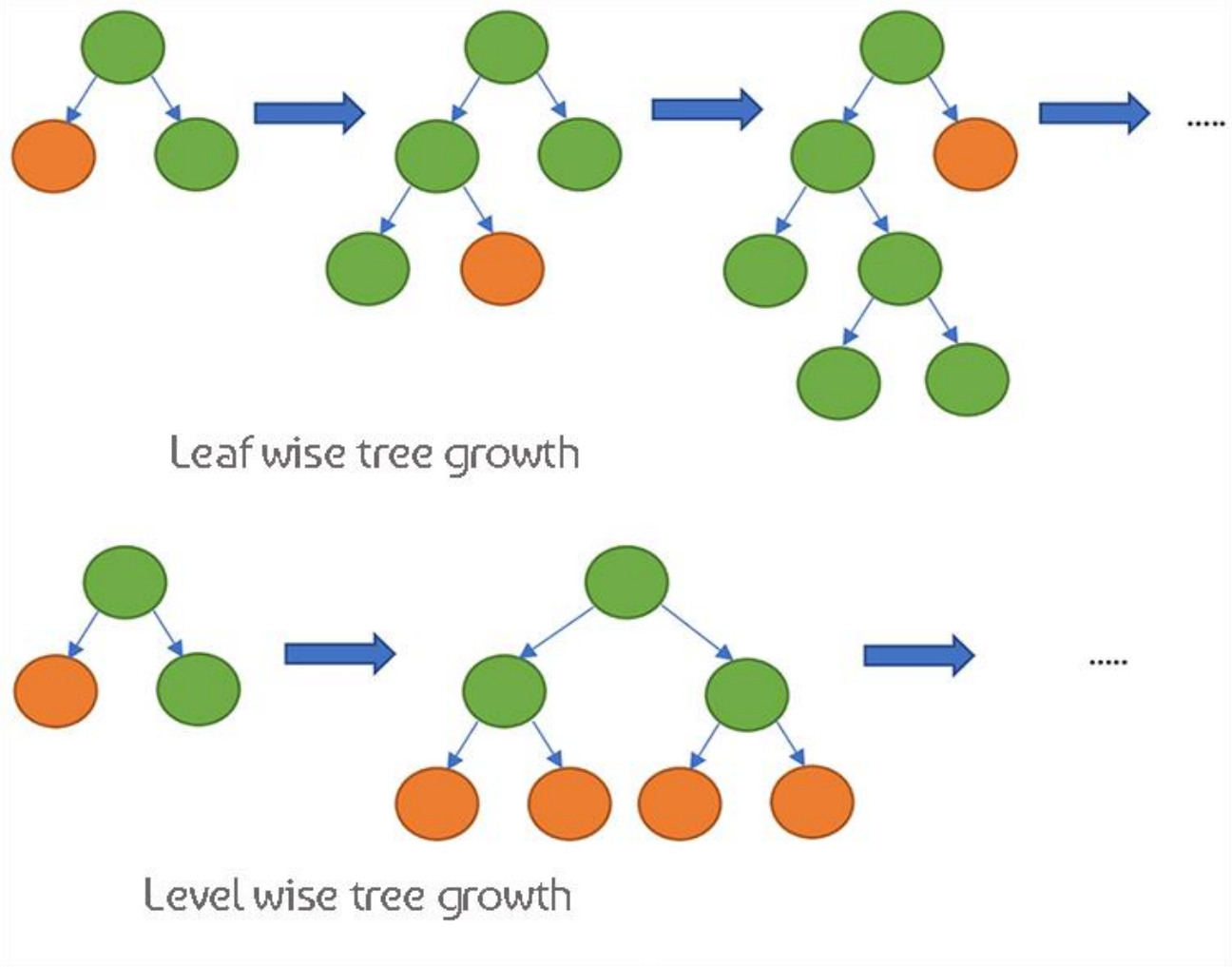
XGBooost e LightGBM : árvores assimétricas
Catboost : árvores simétricas

Desenvolvimento das árvores

LightGBM: cresce as árvores por folhas

XGBoost: cresce por nível

- árvores mais balanceadas
- assimetria pode ocorrer devido a restrições (ex: profundidade)
- exigência de ganho para divisão (melhora na performance).



Desenvolvimento das árvores

O XGBoost expande a árvore nível por nível. Isso significa que ele divide todos os nós de uma mesma profundidade antes de passar para a próxima camada.

O XGBoost faz uma pergunta diferente em cada nó.

Exemplo: No Nível 2, o galho da esquerda pode perguntar "Idade > 30?", enquanto o galho da direita pergunta "Pressão Arterial > 140?".

Nem todos os nós são necessariamente divididos — a divisão só ocorre se houver ganho suficiente. Assim, embora o crescimento seja por nível, a árvore resultante pode ser estruturalmente assimétrica.

O LightGBM escolhe a folha que reduz mais a perda (o erro) e a divide, independentemente do nível em que ela esteja.

Diferenças e similaridades nos hiperparâmetros dos algoritmos de boosting

Função / Hiperparâmetro	XGBoost	CatBoost	LightGBM	Interpretação
Número de árvores	n_estimators	n_estimators	n_estimators	Controlam o número total de árvores/iteração.
Taxa de aprendizado	learning_rate	learning_rate	learning_rate	Controla o passo de aprendizado -> o quanto do erro cada nova árvore vai poder tentar corrigir.
Profundidade máxima da árvore	max_depth	max_depth	max_depth	
Número mínimo de amostras por folha (final)	min_child_weight	min_data_in_leaf	min_data_in_leaf	XGBoost usa peso Heussiano ("nível de atenção" que o modelo dá ao paciente).
Subamostragem de linhas (samples)	subsample	subsample	subsample	Controla proporção de OBS usados para treinar cada árvore
Subamostragem de colunas (features)	colsample_bytree	não disponível	colsample_bytree	Proporção de features usadas em cada árvore.
Regularização L1	reg_alpha	Não exposto diretamente	reg_alpha	L1 “pune” coeficientes grandes: variáveis pouco úteis → coeficiente 0; variáveis importantes → permanecem
Regularização L2	reg_lambda	reg_lambda	reg_lambda	L2 não zera coeficientes, apenas reduz os valores, deixando-os menores.
Tratamento de dados categóricos	Precisa de codificação	Nativo	Precisa de codificação	CatBoost trata dados categóricos internamente; XGBoost e LightGBM exigem preprocessamento.

TabPFN

Arquitetura baseada em Transformers, adaptada para o uso em bancos de dados estruturados.

É um algoritmo pré-treinado a partir de milhões de conjuntos de dados sintéticos.

Os transformers ajudam a **processar todas as colunas de uma tabela ao mesmo tempo**, capturando **interações complexas entre variáveis** de forma eficiente.

Permite predições em bancos de dados pequenos ou médios **rapidamente**, sem precisar de ajuste manual de hiperparâmetros.

Representa uma mudança significativa na modelagem de dados tabulares, superando as abordagens tradicionais.